Neural subgrid fluxes with convex limiting for the shallow-water equations

What we built, the maths behind it, and how training and inference work. Reimplementation of Timofeyev, Schwarzmann & Kuzmin, *Math. Comput. Simul.* 238 (2025) 163–178.

1. The idea

The shallow-water equations describe a water surface (a river, a coastal sea) by two fields: the depth $h(x,t)$ and the discharge $q = hv$ (depth × velocity). Resolving every wave takes a **fine mesh**: here 2000 cells and a time step of 0.005. A **coarse mesh** of 100 cells is 20× smaller and tolerates a much larger time step, but the standard scheme (local Lax–Friedrichs, LLF) is numerically diffusive and smears the waves away.

The paper's remedy: keep the cheap coarse solver and add a **correction flux** $\tilde{G}$ predicted by a small neural network that was trained on fine-mesh data. A network can output anything, including a correction that drives the depth negative, so a limiter (**MCL**, monolithic convex limiting) clips $\tilde{G}$ to a range in which the update is provably safe.

> **One line:** coarse state $U \to$ coarse LLF flux $\bar{F}$ + network correction $\tilde{G} \to$ MCL clips it to $\tilde{G}^* \to$ flux $F = \bar{F} + \tilde{G}^* \to$ time step $\to$ next coarse state.

DNS — direct numerical simulation: the fine 2000-cell run, treated as the truth.

LLF — local Lax–Friedrichs: a robust but diffusive way to compute the flux between two cells.

Bar state — the averaged solution of the two-cell problem; the building block of MCL.

MCL — monolithic convex limiting: clips a correction so bounds such as $h > 0$ hold.

Standardise — shift and scale data to zero mean, unit variance before the network sees it.

Flux — how much h or q crosses a cell interface per unit time.

Stencil — the neighbouring cells a formula looks at (here 4 cells: $i-1, \ldots, i+2$).

CFL condition — the time step must be small enough that no wave crosses a whole cell per step.

Heun — a second-order, two-stage time-stepping method.

Early stopping — stop training when the validation loss stops improving (guards against over-fitting).

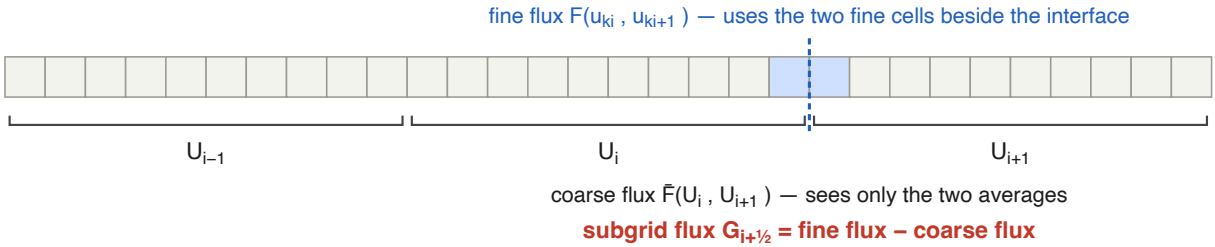


Fig. 1. Fine and coarse mesh (k = 20 fine cells per macrocell in the real setup, 10 drawn). The exact subgrid flux G is what the coarse scheme gets wrong at an interface: the flux the fine solution really carries across it minus the flux the coarse solver computes from its averages alone.

2. The maths, step by step

2.1 Equations

With $u = [h, q]^T$ on a periodic domain $(0, L)$, $L = 100$, $g = 9.812$:

$$u_t + f(u)_x = 0, \qquad f(u) = \left[q,\ \tfrac{q^2}{h} + \tfrac{1}{2}gh^2\right]^T.$$

2.2 The fine scheme (the "truth", DNS)

Finite volumes on $N_f = 2000$ cells ($\Delta x = 0.05$). The LLF flux through interface $i+\frac{1}{2}$, with the largest local wave speed λ:

$$F_{i+1/2} = \tfrac{1}{2}\big(f(u_i) + f(u_{i+1})\big) - \tfrac{1}{2}\lambda_{i+1/2}(u_{i+1} - u_i), \quad \lambda_{i+1/2} = \max\big(|v_i| + \sqrt{gh_i},\ |v_{i+1}| + \sqrt{gh_{i+1}}\big), \quad \tfrac{d}{dt}u_i = \tfrac{F_{i-1/2} - F_{i+1/2}}{\Delta x}.$$

Time stepping is Heun's method (two Euler stages averaged). The *bar state* $\bar{u}_{i+1/2} = \frac{1}{2}(u_i + u_{i+1}) - \frac{f(u_{i+1}) - f(u_i)}{2\lambda_{i+1/2}}$ rewrites each Euler stage as a convex combination of u_i and bar states whenever $\frac{\Delta t}{\Delta x}(\lambda_{i-1/2} + \lambda_{i+1/2}) \le 1$ (the CFL condition), which is why the depth stays positive.

2.3 Coarse variables and the exact subgrid flux

Group $k = 20$ fine cells into a macrocell and average: $U_i = \frac{1}{k}\sum_{j \in \text{cell } i} u_j$ ($N_c = 100$, $\Delta X = 1$). Averaging the fine equation gives an **exact** coarse equation:

$$\tfrac{d}{dt}U_i = \frac{(\bar{F}_{i-1/2} + G_{i-1/2}) - (\bar{F}_{i+1/2} + G_{i+1/2})}{\Delta X}, \qquad G_{i+1/2} = F(u_{ki}, u_{ki+1}) - \bar{F}(U_i, U_{i+1}),$$

where $\bar{F}$ is the same LLF formula applied to the coarse averages. G is "anti-diffusive": coarse LLF smooths, G sharpens it back (we checked this identity to 10^{-16}). The catch: G needs the fine states, which a coarse run does not have.

2.4 Learning G

Approximate $\tilde{G}_{i+1/2} = \mathcal{N}(U_{i-1}, U_i, U_{i+1}, U_{i+2}; \omega)$: four neighbouring cells × (h, q) = 8 inputs, 2 outputs (G^h, G^q), three hidden layers of 128 neurons with leaky-ReLU (slope 0.01), 34,434 parameters, trained on the mean-squared error

$$\mathcal{L}(\omega) = \frac{1}{M} \sum_{j=1}^{M} \left\| \mathcal{N}(U_{i-1}^{(j)}, \ldots, U_{i+2}^{(j)}; \omega) - G_{i+1/2}^{(j)} \right\|_2^2.$$

2.5 Limiting with MCL

Write the corrected coarse update with limited bar states $\bar{U}^{*,\pm}_{i+1/2} = \bar{U}_{i+1/2} \pm \tilde{G}^*_{i+1/2}/\Lambda_{i+1/2}$:

$$\tfrac{d}{dt} U_i = \tfrac{1}{\Delta X} \left[\Lambda_{i-1/2} \left(\bar{U}^{*,+}_{i-1/2} - U_i \right) + \Lambda_{i+1/2} \left(\bar{U}^{*,-}_{i+1/2} - U_i \right) \right].$$

MCL demands a *local maximum principle*: the new bar states must stay within the range of their neighbours, $h_i^{\min} \le \bar{h}^{*,-}_{i+1/2} \le h_i^{\max}$ and $h_{i+1}^{\min} \le \bar{h}^{*,+}_{i+1/2} \le h_{i+1}^{\max}$, with $h_i^{\min\,/\,\max} = \min\,/\,\max(\bar{h}_{i-1/2}, \bar{h}_{i+1/2})$, and the same for the velocity $\bar{v} = \bar{q}/\bar{h}$. It is applied in two steps. **Height first** (eq. 27 of the paper):

$$\tilde{G}^{h,*} = \begin{cases} \min\left(\tilde{G}^h,\ \Lambda \min(\bar{h} - h_i^{\min},\ h_{i+1}^{\max} - \bar{h})\right), & \tilde{G}^h \ge 0 \\ \max\left(\tilde{G}^h,\ \Lambda \max(\bar{h} - h_i^{\max},\ h_{i+1}^{\min} - \bar{h})\right), & \tilde{G}^h < 0 \end{cases}$$

Then discharge: limit $\Delta\tilde{G}^q = \tilde{G}^q - \tilde{G}^{h,*}\bar{v}$ to the analogous velocity interval (eq. 31) and set $\tilde{G}^{q,*} = \tilde{G}^{h,*}\bar{v} + \Delta\tilde{G}^{q,*}$. Under the CFL condition the updated U_i is then a convex combination of admissible states, so $h > 0$ **whatever the network outputs**. If the network is accurate the limits are inactive and MCL changes nothing.

3. How it runs

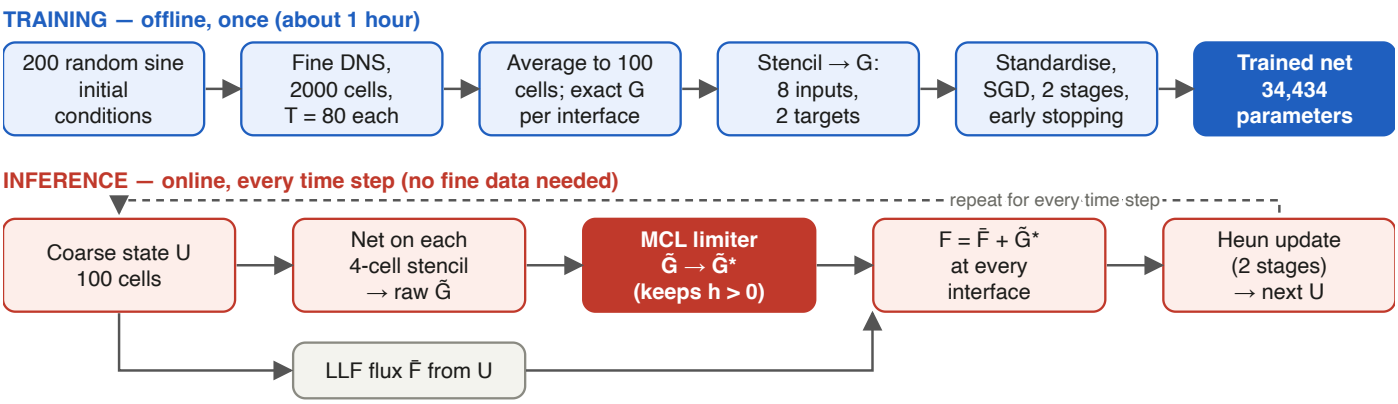


Fig. 2. Training happens once offline; inference needs only the coarse state.

Training

1. **Initial conditions.** 200 random sine waves: $h = 2 + A_h \sin(2\pi k_h x/L + \varphi_h)$, $v = V_0 + A_v \sin(2\pi k_v x/L + \varphi_v)$, with $V_0 \sim U(1,2)$, $A \sim U(0.2, 0.6)$, $k \in \{1..6\}$, $\varphi \sim U(0, 2\pi)$.
2. **Labels.** Run the fine DNS to $T = 80$. Every 10th step, average to 100 cells and compute the exact G at all 100 interfaces: 32 million (stencil, G) pairs. (The paper keeps every step, 320 M.)
3. **Standardise** inputs and outputs (zero mean, unit variance); the last 20 % of trajectories are held out for validation.
4. **Optimise.** SGD, batch 128. Stage 1: learning rate 0.05, up to 500 epochs; stage 2: 0.001, up to 2000; early stopping in both. Ours stopped after 32 and 18 epochs, validation loss $3.96 \cdot 10^{-4}$ (standardised units).

Inference (each time step)

1. **Start.** Box-average the fine initial condition to the 100-cell state U; the fine solver is never used again.
2. **LLF flux** $\bar{F}$ at the 100 interfaces from U.
3. **Network.** Build each 4-cell stencil, standardise, evaluate $\mathcal{N}$, undo the standardisation: raw $\tilde{G}$.
4. **MCL** (optional switch): $\tilde{G} \to \tilde{G}^*$ by the two limiters of §2.5.
5. **Update.** $F = \bar{F} + \tilde{G}^*$, $\dot{U}_i = (F_{i-1/2} - F_{i+1/2})/\Delta X$. Heun's method repeats steps 2–5 for its two stages. The time step is the paper's 0.005, or up to 0.1.

4. What we get

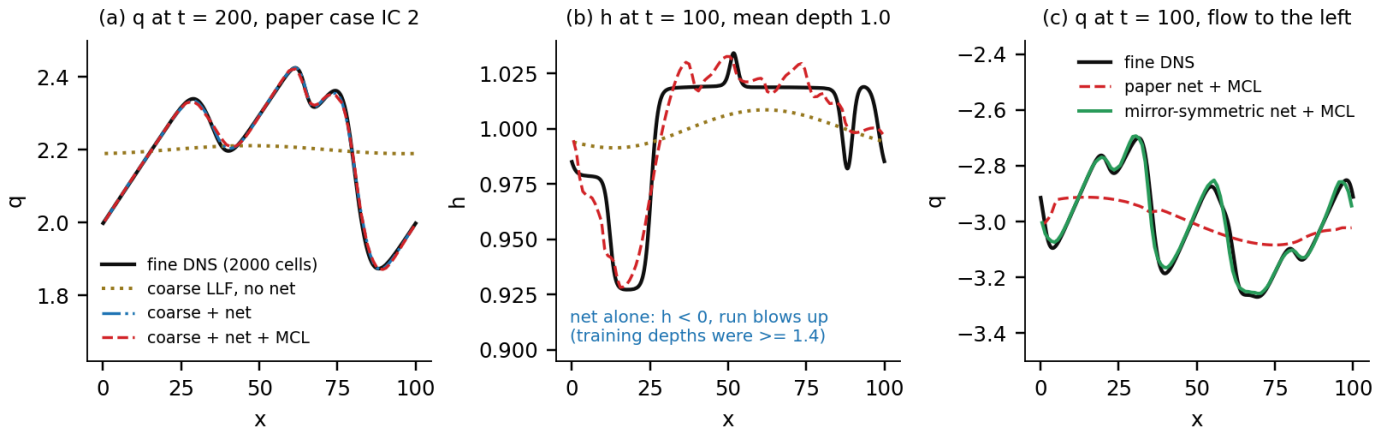


Fig. 3. (a) A 100-cell model with the net follows the 2000-cell truth; plain coarse LLF flattens it. (b) Far outside the training range the net alone blows up; MCL keeps the run alive. (c) Training only contained flow to the right; building mirror symmetry into the network fixes flow to the left.

Table 1. Relative ℓ^2 error of q against the fine solution averaged to 100 cells, median per group of the 20 test cases (paper cases, outside training range, tough, where the method breaks).

model	params	paper cases	outside training	tough	breaks	all	net alone blows up
plain coarse LLF (no net)	–	2.57%	5.51%	16.81%	19.63%	**7.78%**	–
paper net 3×128 + MCL	34,434	0.43%	2.45%	8.13%	13.61%	**2.94%**	5 of 20
mirror-symmetric 3×128 + MCL	34,434	0.58%	1.71%	3.80%	9.64%	**2.41%**	3 of 20
small net 3×32 + MCL	2,466	0.56%	1.77%	4.98%	10.07%	**2.52%**	5 of 20

- **It works as the paper claims.** On the paper's own cases the 100-cell net + MCL is 0.3–0.5 % off in q against 1.5–7 % for plain coarse LLF, and it keeps working at mean depths ± 25 % from the training value and for runs 5× longer than the training trajectories ($t = 400$ against 80, q error 0.4 %).
- **MCL is a safety net, not an accuracy booster.** Without it the net alone blows up in 5 of 20 cases (very shallow water, near-dry troughs, strong dam breaks); with it, in none. Errors there stay large (dam breaks 5–26 %) because the network never saw shocks.
- **MCL can also hurt.** For waves resolved by only 5–7 coarse cells, clipping flattens real crests: q error 15 % with MCL against 2 % without.
- **Our addition (not in the paper).** A mirror-symmetric network, with a built-in $x \to -x$ symmetry that the exact G obeys: it fixes left-moving flow (4.3 % → 0.7 %) and cuts dam-break errors by 40–80 %.
- **From the paper: smaller networks.** The paper also trains 3×64, 3×32 and 3×16 networks and finds 32 neurons often as good as 128. We trained only 3×32 (on half the data) and agree: with MCL its median error is 2.5 % against 2.9 % for the 3×128 network, with 14× fewer parameters.
- **The speed-up is modest.** One network call costs about 90 µs in PyTorch, more than a whole 2000-cell DNS right-hand side in NumPy (about 60 µs). At the paper's $\Delta t = 0.005$ the reduced models are therefore *slower* than the DNS here; with $\Delta t = 0.05$ they are about 2–5× faster (plain coarse LLF: 17×).

Differences from the paper. Training uses every 10th time step (memory). Our Fig. 6 Lax–Wendroff baseline is less diffusive than the paper's (their variant is not specified). The network-size study (their Figs. 9–10) is only partly reproduced: we trained 3×128 and 3×32 but not 3×64 or 3×16. An earlier version of our code built the training target G from the wrong fine cell; this report uses the corrected code (a test checks the exact-G identity of §2.3).

5. Where things are

module (`work/final_btp/swe/`)	implements
`physics.py`, `solver.py`	§2.1–2.2: flux, LLF, bar states, Heun
`data.py`	§2.3 and training data: exact G, stencil
`network.py`, `training.py`	§2.4: network, SGD, early stopping
`mcl.py`	§2.5: height and discharge limiters
`tests/`	33 checks against the paper's equations

Run it (from `work/final_btp`):

`python scripts/generate_data.py` → `python scripts/train.py` → `python scripts/precompute.py` → `python dashboard/server.py`, then open `localhost:8000`. The dashboard shows any of 20 cases (or your own upload) for all three networks side by side, with the comparison matrix of Table 1.